

Parallel A* Password Recovery

Sequential, Pthreads, OpenMP, OpenCilk, CUDA and Hybrid CPU+GPU Implementations

Parallel & Distributed Systems (Homework 4)
Apostolos Pantazis (10910)

September 2026

Abstract

This report presents an MD5 password-recovery tool formulated as an **A* state-space search** over a prefix tree and parallelized across five paradigms: POSIX threads, OpenMP, OpenCilk, CUDA, and a Hybrid Pthreads + CUDA design. The search uses a uniform cost model over a fixed 76-symbol alphabet with a bounded beam to cap memory. We describe the algorithmic formulation, the parallelization strategy of each implementation, and an experimental evaluation on a 28-core host equipped with an NVIDIA GeForce RTX 4070 Ti SUPER. All implementations recover length-4 targets correctly; the Hybrid design additionally succeeds on the harder length-5 space where the CPU-only variants exhaust their search budget (provided it runs with 2 or more threads).

1 Problem Formulation

Given the MD5 digest of an unknown password of known length L , the goal is to recover the plaintext. Because cryptographic hashes exhibit the *avalanche effect*, there is no continuous notion of “distance” between a partial candidate’s hash and the target. We therefore do not use the hash to guide the search; instead we treat the problem as a graph search over a prefix tree and use A* [1, 2] as a systematic, memory-bounded traversal.

State-space model.

- **State:** a candidate prefix string s at depth d .
- **Alphabet:** a fixed 76-symbol set — lowercase, uppercase, digits, and symbols `!@#$%^&*()-_+=.`
- **Path cost:** $g(n) = d \cdot \text{EDGE_COST}$, with $\text{EDGE_COST} = 1.0$.
- **Heuristic:** $h(n) = (L - d) \cdot \text{EDGE_COST}$ (remaining depth; admissible).
- **Evaluation:** $f(n) = g(n) + h(n) = L$, uniform across all candidates of the target length.
- **Goal test:** MD5 verification is performed *only* when a candidate reaches depth L (a leaf).

Since f is uniform, A* here behaves as a systematic frontier expansion. To bound memory we apply **beam trimming**: when the priority queue exceeds $\text{BEAM_CAP} = 3\,000\,000$ states, a **quickselect** keeps the best $\text{BEAM_KEEP} = 1\,200\,000$ and re-heapifies. If the target is not recovered within the allocated node or time budget, the search terminates with failure.

2 Software Architecture

The project shares a common core across all targets:

File	Role
<code>astar_heap.{h,c}</code>	Min-heap priority queue + beam-trim (quickselect + heapify).
<code>md5.{h,c}</code>	Standalone RFC 1321 MD5 for CPU targets.
<code>crack_astar.c</code>	Sequential baseline.
<code>crack_astar_pthreads.c</code>	POSIX threads, atomic dynamic task queue.
<code>crack_astar_omp.c</code>	OpenMP dynamic loop scheduling.
<code>crack_astar_opencilk.c</code>	OpenCilk <code>cilk_for</code> work-stealing.
<code>crack_astar_cuda.cu</code>	CPU-driven heap + GPU batch leaf hashing.
<code>crack_astar_hybrid.cu</code>	Pthreads + per-thread CUDA streams.

Priority queue. A binary min-heap ordered by f supports `push` and `pop` in $O(\log n)$. Beam trimming uses `quickselect` to partition the best k states in expected $O(n)$ before re-heapifying, avoiding a full $O(n \log n)$ sort.

3 Parallelization Strategies

3.1 Pthreads

The root search space is partitioned into two-character prefix tasks ($76^2 = 5776$ independent sub-searches for $L \geq 2$). Worker threads claim tasks dynamically through an atomic counter (`atomic_fetch_add`), which provides natural load balancing. Each thread owns a *private* priority queue, eliminating lock contention on the heap. An atomic flag signals early termination the moment any thread finds the match.

3.2 OpenMP

Uses the same 5776-task decomposition, distributed with `#pragma omp parallel for schedule(dynamic, 1)`. A per-thread queue is preallocated and reused (indexed by `omp_get_thread_num()`) to avoid repeated heap allocation.

3.3 OpenCilk

Exposes the sub-tasks to the work-stealing runtime with `cilk_for`; each iteration runs an independent A* traversal with its own local heap. Worker count is controlled strictly via the `CILK_NWORKERS` environment variable.

3.4 CUDA

The CPU performs the pointer-based traversal and heap operations while completed leaf candidates accumulate in a batch buffer. When the batch fills (or a node threshold is reached) the candidates are copied to the GPU and hashed in parallel by an optimized MD5 kernel. Here `-threads` denotes the GPU *batch size*, not a CPU thread count.

3.5 Hybrid Pthreads + CUDA

Combines CPU dynamic task distribution with GPU batch execution [3]. Each POSIX worker is assigned (i) its own CUDA stream (`cudaStreamCreate`) for concurrent GPU execution, (ii) pinned host memory (`cudaHostAlloc`) for DMA transfers, and (iii) private device buffers/flags to avoid device-side synchronization [4].

4 Build & Run

```

make all                                     # sequential, pthreads, openmp
make crack_astar_opencilk CILK_CC=/opt/opencilk/bin/clang
make crack_astar_cuda NVCC_ARCH=sm_89      # match GPU compute capability
make crack_astar_hybrid NVCC_ARCH=sm_89

./crack_astar 4 PtYg
./crack_astar_pthreads 4 PtYg --threads 8
./crack_astar_hybrid 5 yJuq\ ) --threads 16 --batch-size 262144

```

5 Experimental Setup

CPU	28 logical cores (WSL2, Linux 6.6)
GPU	NVIDIA GeForce RTX 4070 Ti SUPER (sm_89)
Compilers	gcc -O2 -std=c11; OpenCilk clang; nvcc -O3
Alphabet	76 symbols; EDGE_COST=1.0
Beam	BEAM_CAP=3M, BEAM_KEEP=1.2M
Test set	six random length-4 targets; four length-5 (exploratory)

Timings are wall-clock seconds for the A* phase. Because thread task order affects which leaf is reached first, node counts vary slightly across thread counts — this is inherent to a nondeterministic parallel frontier and does not affect correctness.

6 Results

6.1 CPU scaling on length-4 targets

Table 1 lists representative wall-clock times. Sequential timings vary strongly with the target because A* frontier ordering reaches some targets much sooner than others. The Pthreads and OpenMP variants scale near-linearly with thread count. OpenCilk’s timing is noticeably more variable across different targets (e.g., very fast on `j&mU`, but slower on `el31`) because its work-stealing scheduler inherently alters the order in which the search tree leaves are explored.

Table 1: Wall-clock time (s) on length-4 targets. *seq* is single-threaded; parallel columns use the stated thread count. CUDA uses batch size 262144.

Target	seq	pth-8	pth-32	omp-8	omp-32	cilk-8	cuda
PtYg	0.362	0.253	0.097	0.593	0.200	0.162	0.130
j&mU	1.236	0.060	0.027	0.145	0.051	0.021	0.423
=h#B	2.426	0.483	0.184	1.029	0.360	0.383	0.891
el31	3.244	0.027	0.012	0.062	0.022	0.228	1.054
iEl(	3.929	0.054	0.022	0.117	0.041	0.363	1.156
2h-p	3.876	0.342	0.130	0.769	0.249	0.328	1.364

6.2 Pthreads speedup

Table 2 reports the Pthreads speedup over its own single-thread run for the `PtYg` target. Scaling shows super-linear speedup (> 100% efficiency) at 4 and 8 threads. This occurs because parallel work-stealing allows the target leaf to be found out-of-order much earlier than a single-thread search would reach it. Efficiency eventually drops (e.g., 68% at 32 threads) due to parallel overhead and search saturation.

Table 2: Pthreads self-relative speedup for target PtYg (baseline: pthreads with 1 thread, 2.106 s).

Threads	Time (s)	Speedup	Efficiency
1	2.106	1.00×	100%
2	1.075	1.96×	98%
4	0.520	4.05×	101%
8	0.253	8.32×	104%
16	0.150	14.04×	88%
32	0.097	21.71×	68%

6.3 Hybrid: length-4 sweeps

On length-4 targets the Hybrid design is competitive with the best CPU variants, with a sweet spot around 4–8 threads and batch size 262 144 (Table 3). Beyond that, GPU launch overhead and increased frontier exploration erode the benefit.

Table 3: Hybrid thread sweep, batch size 65 536, target PtYg (length 4).

Threads	Time (s)	Nodes	GPU batches
1	0.460	242 005	277
2	0.238	242 329	277
4	0.133	242 506	276
8	0.085	245 048	277
16	0.100	256 689	288
32	0.159	444 685	524

6.4 Hybrid: Evaluation on length-5 targets

Length-5 represents a significantly larger search space ($76^5 \approx 2.53 \times 10^9$ states). The CPU-only implementations exhaust their search budget on length 5, and even the Hybrid design requires multiple threads to succeed within the 60-second budget (failing with 1 thread but consistently recovering all targets with 2 or more). Table 4 shows the batch-size sweep at 16 threads: larger batches amortize GPU launch overhead and reduce overall execution time.

Table 4: Hybrid batch-size sweep, 16 threads, length-5 targets. All *found*.

Target	Batch	Time (s)	Nodes	GPU batches
yJuq)	16 384	2.914	10 866 295	49 734
	65 536	2.431	10 878 084	12 443
	262 144	2.322	10 903 192	3 111
	1 048 576	2.328	10 947 245	777
ntG0y	16 384	1.555	5 907 112	27 035
	65 536	1.318	5 910 361	6 758
	262 144	1.289	5 920 987	1 689
	1 048 576	1.324	5 916 686	414
(K5cq	16 384	8.005	31 331 640	143 419
	65 536	6.836	31 383 225	35 908
	262 144	6.554	31 360 143	8 965
	1 048 576	6.522	31 503 116	2 247
=e!4f	16 384	8.595	32 953 449	150 842
	65 536	7.179	32 948 742	37 699
	262 144	6.935	32 964 065	9 423
	1 048 576	6.911	33 017 988	2 355

7 Discussion

- **Decomposition quality.** The 5 776 two-character prefix tasks provide sufficient granularity for load balancing on a 28-core host, while per-thread private heaps eliminate lock contention on the priority queue, allowing near-linear scaling up to 8 threads.
- **Runtime differences.** OpenMP is consistently a bit slower than Pthreads at the same thread count due to scheduling overhead. OpenCilk’s work-stealing makes its runtime highly target-dependent — sometimes it is the fastest, while other times it is slower than Pthreads at the same worker count, making the number of expanded nodes less predictable.
- **GPU trade-off.** Pure CUDA rarely beats a well-parallelized CPU at length 4 because the bottleneck is CPU-side frontier expansion, not hashing. The GPU only pays off when the leaf-verification volume is large, which is exactly the length-5 regime.
- **Batch size matters.** For the Hybrid design, small batches (16 384) drown in kernel-launch overhead; batches of 262 144–1 048 576 minimize launches and give the best time.
- **Oversubscription.** At 32 threads several Hybrid cases show a jump in node count and time — contention for the single GPU plus frontier divergence outweighs added CPU parallelism.

8 Verification

Automated functional tests (`scripts/run_verification_tests.sh`) confirm every built implementation recovers the reference target `PtYg`. The four CPU binaries compile cleanly under `-Wall -Wextra` with no warnings, while the CUDA binaries compile cleanly with `nvcc`.

Table 5: Automated verification results for all implementations.

Implementation	Result
Sequential	PASS — recovered <code>PtYg</code>
Pthreads	PASS — recovered <code>PtYg</code>
OpenMP	PASS — recovered <code>PtYg</code>
OpenCilk	PASS — recovered <code>PtYg</code>
CUDA	PASS — recovered <code>PtYg</code>
Hybrid	PASS — recovered <code>PtYg</code>

9 Conclusion

All six implementations are correct and reproducible. On length-4 targets the CPU parallel variants achieve near-ideal speedups (Pthreads reaches $\approx 21\times$ at 32 threads), while pure GPU offloading is limited by CPU-side frontier expansion. The decisive result is at length 5: only the Hybrid Pthreads + CUDA design, combining dynamic CPU task distribution with per-thread CUDA streams and large GPU batches, extends the reach of the search into a space where every CPU-only implementation fails. This confirms that heterogeneous CPU+GPU cooperation — not GPU offloading alone — is what unlocks the harder search regime.

You can find all the necessary files for this project on this GitHub repository:

<https://github.com/pantazisa/Homework>

References

- [1] P. E. Hart, N. J. Nilsson, and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

- [2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.
- [3] J. Nickolls, I. Buck, M. Garland, and K. Skadron, “Scalable Parallel Programming with CUDA,” *Queue*, vol. 6, no. 2, pp. 40–53, 2008.
- [4] NVIDIA Corporation, *CUDA C++ Programming Guide*, Version 12.0, 2023.